

5-Day Learning Log

Matplotlib, A* Search, and the UAS-DTU Path Planning Project

Day 1 — Getting started with Matplotlib

I started off just getting comfortable with the basics of matplotlib since I hadn't touched it in a while. The first thing I had to figure out was the backend issue — since I'm working in a browser-based Python environment (NextLeap) without a display, regular matplotlib windows don't work. Setting **matplotlib.use("Agg")** before importing pyplot fixed that, since Agg renders straight to an image file instead of trying to open a GUI window.

After that I went through the usual plotting basics — **plt.plot()**, **plt.scatter()**, and **plt.imshow()** for displaying 2D array data as an image. I also played around with **plt.figure(figsize=...)** to control the output size, and the standard labeling functions (**plt.title()**, **plt.xlabel()**, **plt.ylabel()**, **plt.grid()**).

The last piece was learning to save figures instead of showing them — **plt.savefig(filename, dpi=150)** followed by **plt.close()**. I didn't think much of **plt.close()** at first, but it matters once you're generating more than one plot in a script, otherwise figures pile up in memory.

Practice: made a few throwaway plots of small 2D arrays with **imshow()**, and tried **plt.text()** with **ha="center"**, **va="center"** to label individual cells — this ended up being exactly what I needed later for labeling Start and Goal on the grid.

Day 2 — Using Matplotlib for grid visualization

Today was more about applying yesterday's basics to something closer to what I'd actually need — visualizing a grid, since that's the format the path-planning problem uses.

The grid itself is just text: '.' for free space, '#' for obstacles, 'S' for start, 'G' for goal. That's not something **imshow()** can plot directly, so I converted it into a numeric grid instead — 0 for free, 1 for obstacle, 2 for an explored cell, 3 for the final path, 4 for start, 5 for goal. Once it's numbers, **imshow()** colors each value automatically.

I also used **plt.xticks()/plt.yticks()** to line up gridlines with each cell, and **plt.tight_layout()** so labels don't get cut off at the edges of the figure.

Since I knew I'd need to generate a picture for more than one test grid later, I wrote this as a small reusable function rather than one-off code — it takes a grid, an explored list, and a path, and produces one labeled image. Saved me from rewriting the same plotting logic three times on Day 5.

Day 3 — Learning how A* actually works

Before writing any code I wanted to actually understand A*, not just copy an implementation, so today was mostly reading and working through the logic on paper.

The core idea is the cost function $f(n) = g(n) + h(n)$, where $g(n)$ is the real cost from the start to node n , and $h(n)$ is a heuristic estimate of the remaining cost from n to the goal. A* always expands the node with the lowest $f(n)$ next, using a priority queue.

Since the project only allows up/down/left/right movement (no diagonals), the right heuristic here is Manhattan distance: $h(n) = |\text{row1} - \text{row2}| + |\text{col1} - \text{col2}|$. This matters because Manhattan distance never overestimates the real cost when diagonal movement isn't allowed — which makes it admissible, and that's what guarantees A* actually finds the shortest path instead of just a decent one.

I also had to get clear on the bookkeeping: `g_score` keeps the cheapest known cost to reach each cell, `came_from` stores each cell's parent so the path can be rebuilt at the end, and a `closed_set` keeps track of cells that are already finalized so they aren't reprocessed.

One detail that confused me at first: in the heap you push `(f_score, counter, position)` instead of just `(f_score, position)`. The counter is just a tie-breaker — if two cells have the same `f_score`, Python would otherwise try to compare the position tuples directly, which can error out or behave unpredictably.

Spent the rest of the day tracing A* by hand on a small 4x4 grid, writing out g , h , and f for each cell as it got expanded, just to make sure I actually understood it before trusting an implementation.

Day 4 — Implementing A* in Python

With the theory settled, today was about turning it into actual code. Rather than writing one big function, I broke it into smaller pieces so each part could be tested on its own:

heuristic(a, b) — the Manhattan distance calculation from Day 3.

get_neighbors(position, grid) — returns the valid up/down/left/right neighbors of a cell, filtering out anything outside the grid boundaries or blocked by an obstacle ('#').

find_start_goal(grid) — scans the grid for 'S' and 'G' and raises a `ValueError` if either is missing, so the algorithm fails loudly instead of silently misbehaving.

reconstruct_path(came_from, current) — walks backward from the goal through the `came_from` chain, then reverses the list so the final path reads Start → Goal.

The **astar(grid)** function itself ties everything together: it pushes the start node onto the heap, pops the lowest- f node each loop, skips it if already closed, checks whether it's the goal, and otherwise expands its neighbors — updating `g_score` and `came_from` whenever a cheaper route to a neighbor is found. Every move costs 1, so $g(n)$ ends up just being the step count from the start.

I also timed execution with `time.perf_counter()` rather than `time.time()`, since it's more precise for something that runs in a few milliseconds — `time.time()` isn't granular enough to give a meaningful number on a search this fast.

Before wiring this into anything visual, I ran it on a small hand-made grid and printed out `g_score` and the open set at each step just to sanity-check that the algorithm was behaving the way I'd traced it out on paper the day before.

Day 5 — Putting it all together: the UAS-DTU project

Today was about combining everything from the last four days into one finished script — this is the actual submission for the UAS-DTU technical round (Avionics track): A* path planning with a visual output.

The final script loads a grid (rows of '.', '#', 'S', 'G'), runs it through the A* implementation from Day 4, prints a results summary (path found or not, path cost, number of nodes explored, execution time, and the final coordinate sequence), and then generates a color-coded image of the search using the matplotlib approach from Day 2.

I tested it against three scenarios. Test case 1 is a normal grid with scattered obstacles where a path clearly exists. Test case 2 is a grid where a solid wall of obstacles completely separates start from goal — this was specifically to check that the algorithm reports 'no path found' correctly instead of hanging or crashing. Test case 3 is a larger 7×11 grid with multiple obstacle clusters, mainly to see how it performs and looks visually at a bigger scale.

Getting test case 2 to behave properly mattered more than I expected — it's easy to write pathfinding code that works fine when a path exists but does something weird (infinite loop, exception, wrong output) when one doesn't. A* naturally handles this since the open set eventually empties out with the goal never reached, and the function returns `None` for the path, but I still wanted to see it happen correctly rather than just assume it.

By the end of the day the whole pipeline works start to finish: grid in, A* search, printed report, saved PNG. Keeping the search logic (astar) separate from the reporting (`print_results`) and the plotting (visualize) made it easy to just loop over three different grids in `main()` without duplicating any code.

Summary

Day	Focus	What I came away with
1	Matplotlib basics	Comfortable with figures, saving plots, the Agg backend
2	Grid visualization	A reusable way to plot a numeric grid with <code>imshow()</code>

3	A* theory	Understood $f(n)=g(n)+h(n)$, Manhattan distance, priority queues
4	A* implementation	A working, modular A* function in Python
5	Full project	Complete, tested A* path planner with visual output (UAS-DTU)